



Database Management Systems

PROJECT REPORT

**Design and Implementation of a Database Management System
for Student Attendance and Leave Approval System**

Submitted by

Team Member	Roll Number
Shaurya Mittal	25WU0101128
Samarth Samanekar	25WU0101119
Vanshika Rao	25WU0701027

Academic Year 2026–2027

Abstract

Attendance and leave are normally handled on paper registers and informal approvals. This causes calculation errors, lost leave applications, and disputes over whether a student meets the minimum attendance requirement. This project designs and implements a relational database that records session-wise attendance and runs every leave application through a two-level approval, first the student's mentor and then the Head of Department.

The database has ten normalized tables in Third Normal Form, enforced with primary keys, foreign keys, unique and check constraints. A stored procedure validates leave applications, two triggers enforce the approval order and automatically mark attendance as On Leave when a leave is finally approved, and views report attendance percentage and pending approvals. The database is implemented in MySQL 8, and a web front end demonstrates the complete workflow for students and faculty.

Keywords: DBMS, MySQL, attendance management, leave approval, triggers, stored procedures, normalization.

Contents

1. Introduction
 2. Problem Statement
 3. Objectives
 4. Scope and Requirements
 5. System Design
 6. Normalization
 7. Database Implementation
 8. Workflow of the System
 9. User Interface
 10. Testing and Results
 11. Conclusion and Future Scope
- References
- Appendix A: Report Queries

1. Introduction

A college must track whether each student attends enough classes, and must also decide fairly when a student asks for time off. In many institutions both tasks are done separately and by hand. The attendance register records absences without knowing whether the absence was approved, and the leave application is a paper slip that moves between offices.

A Database Management System (DBMS) removes this separation. Storing attendance and leave in one database means a rule such as “an approved leave must not reduce attendance” can be enforced by the database itself instead of depending on someone remembering to apply it. This project builds such a system for a university department.

2. Problem Statement

Design and implement a database for a Student Attendance and Leave Approval System that:

- stores students, faculty, courses, enrollments, and class sessions without redundancy;
- records attendance for every student in every class session;
- accepts leave applications and routes each one through Mentor and HOD approval;
- adjusts attendance automatically when a leave is approved; and
- reports attendance percentages and identifies students below the 75% requirement.

The problems with the manual process are:

- Attendance percentages are calculated by hand and often wrongly.
- Approved leave is not reflected in the register, so students are wrongly shown as having a shortage.
- There is no record of who approved a leave, when, and with what remarks.
- Overlapping or excessive leave requests are not detected.

3. Objectives

1. To design an Entity–Relationship model and a normalized relational schema for the system.
2. To implement the schema in MySQL with proper integrity constraints.
3. To implement a two-level (Mentor, then HOD) leave approval workflow in the database.
4. To use triggers and a stored procedure so that business rules are enforced inside the database.
5. To provide views and queries that report attendance percentage, shortages, and pending leaves.
6. To build a user interface that demonstrates the complete workflow.

4. Scope and Requirements

4.1 Users

User	Responsibilities
Student	View own attendance, apply for leave, track and cancel requests.
Faculty (Mentor)	Mark attendance for classes; approve or reject leave of mentee students as the first level.
Head of Department	Give the final approval or rejection after the mentor has approved.

4.2 Functional requirements

- Maintain master data: departments, faculty, students, courses, leave types.
- Enroll students in courses and schedule class sessions.
- Mark each student Present, Absent, or On Leave for each session.
- Apply for leave; reject overlapping requests and requests that exceed the semester limit for that leave type.
- Allow the HOD to act only after the mentor has approved.
- On final approval, change the student's attendance in the leave period to On Leave.
- Compute attendance percentage per course and overall, and list students below 75%.

4.3 Non-functional requirements

- **Integrity:** invalid data (for example a leave ending before it starts) must be rejected by the database.
- **Auditability:** every approval decision is stored with approver, level, remarks, and time.
- **Consistency:** no duplicate attendance entry for the same student and session.
- **Performance:** indexes on the columns used most in queries.

4.4 Software used

Component	Tool
DBMS	MySQL 8.0
Database client	MySQL Workbench / mysql command-line client
Front end	HTML, CSS and JavaScript web page
Diagrams	Entity–Relationship diagram of the final schema

5. System Design

5.1 Entities and relationships

The system has ten entities. Two many-to-many relationships are resolved with link tables: Student and Course through Enrollment, and Class Session and Student through Attendance.

Relationship	Cardinality / Meaning
Department – Faculty, Student, Course	1 : N. A department has many faculty, students and courses.
Faculty (mentor) – Student	1 : N. Each student has one mentor; a mentor has many mentees.
Faculty – Course	1 : N. A faculty member teaches many courses.
Student – Course	M : N through Enrollment.
Course – Class Session	1 : N. A course has many sessions.
Class Session – Student	M : N through Attendance, which stores the status.
Student – Leave Request	1 : N. A student can raise many requests.
Leave Type – Leave Request	1 : N. Each request belongs to one leave type.
Leave Request – Leave Approval	1 : N (at most 2). One decision per level: Mentor and HOD.

5.2 ER diagram

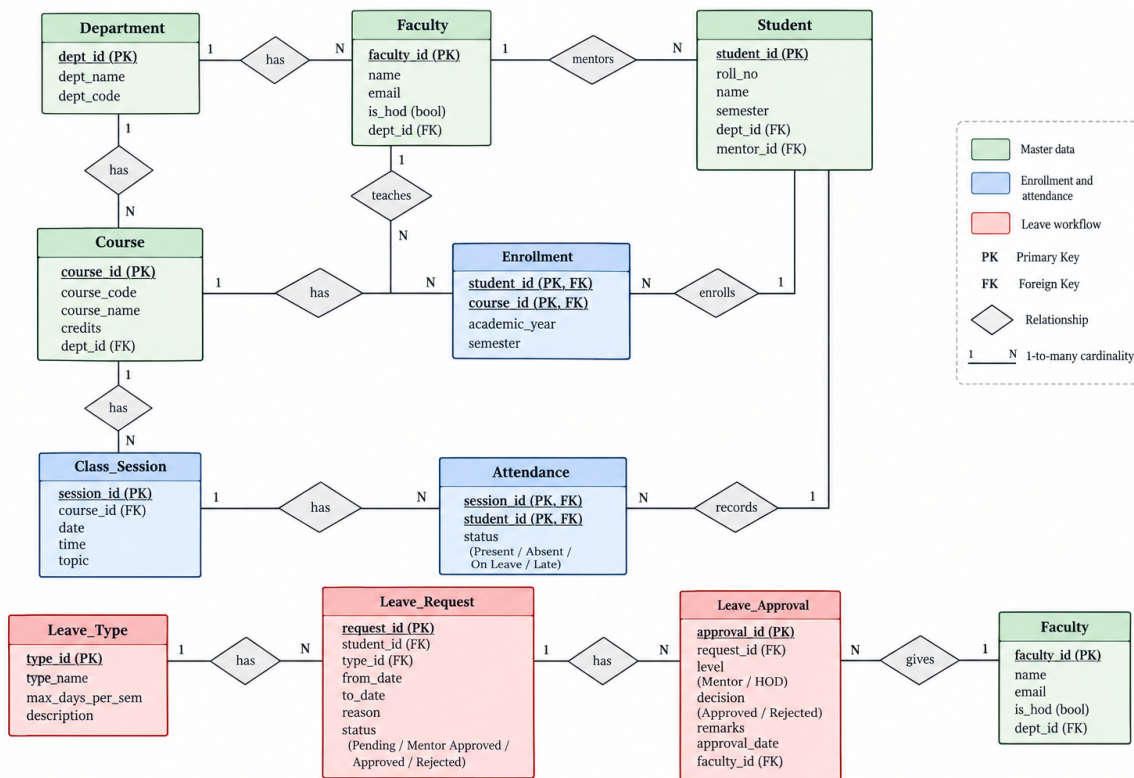


Figure 5.1: ER diagram of the system. Master data is shown in green, enrollment and attendance in blue, and the leave workflow in red. Faculty is repeated at the bottom right to show the “gives” relationship with Leave_Approval.

5.3 Relational schema

Primary keys are underlined in the figure. The relational schema, with foreign keys marked, is:

DEPARTMENT(dept_id, dept_name)

FACULTY(faculty_id, full_name, email, dept_id*, is_hod)

STUDENT(student_id, roll_no, full_name, email, phone, dept_id*, semester, mentor_id*)

COURSE(course_id, course_code, course_name, credits, dept_id*, faculty_id*)

ENROLLMENT(enrollment_id, student_id*, course_id*, academic_year)

CLASS_SESSION(session_id, course_id*, session_date, start_time, end_time, topic)

ATTENDANCE(attendance_id, session_id*, student_id*, status, marked_by*, marked_at)

LEAVE_TYPE(leave_type_id, type_name, max_days_per_sem)

LEAVE_REQUEST(request_id, student_id*, leave_type_id*, from_date, to_date, reason, status, applied_on)

LEAVE_APPROVAL(approval_id, leave_id*, approver_id*, level, decision, remarks, decided_on)

* marks a foreign key.

5.4 Data dictionary of the main tables

Table	Column	Description and constraint
student	roll_no	Unique roll number; NOT NULL
student	semester	CHECK between 1 and 8
student	mentor_id	FK to faculty; mentor who gives first-level approval
attendance	status	ENUM: Present, Absent, On Leave
attendance	(session_id, student_id)	UNIQUE; one record per student per session
class_session	end_time	CHECK end_time > start_time
leave_request	to_date	CHECK to_date >= from_date
leave_request	status	ENUM: Pending, Mentor Approved, Approved, Rejected, Cancelled
leave_type	max_days_per_sem	Maximum leave days of this type per semester
leave_approval	level	ENUM: Mentor, HOD
leave_approval	(leave_id, level)	UNIQUE; one decision per level per request

6. Normalization

First Normal Form (1NF). All attributes hold single atomic values, there are no repeating groups, and every table has a primary key. For example, a student's several courses are not stored in one column; they are separate rows in Enrollment.

Second Normal Form (2NF). There are no partial dependencies. In the link tables Enrollment and Attendance, the non-key attributes depend on the whole key. The attendance status depends on both the session and the student, not on either alone.

Third Normal Form (3NF). There are no transitive dependencies. Department names, course details, and the leave limit of each leave type are stored once in their own tables and referenced by key. A leave request stores only leave_type_id, not the type name or its limit, so changing a limit is a single update.

As a result, there is no update, insertion, or deletion anomaly in the schema. A mentor's name is stored once in Faculty, and changing it does not require touching any student row.

7. Database Implementation

The complete script is in the file attendance_leave_system.sql. It creates the database, tables, indexes, views, procedure, triggers, sample data, and the report queries. The main parts are shown below.

7.1 Table creation (DDL)

The two tables at the centre of the leave workflow:

```
CREATE TABLE leave_request (
    leave_id      INT AUTO_INCREMENT PRIMARY KEY,
    student_id    INT NOT NULL,
    leave_type_id INT NOT NULL,
    from_date     DATE NOT NULL,
    to_date       DATE NOT NULL,
    reason        VARCHAR(255) NOT NULL,
    status        ENUM('Pending','Mentor Approved','Approved',
                       'Rejected','Cancelled') NOT NULL DEFAULT 'Pending',
    applied_on    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CHECK (to_date >= from_date),
    FOREIGN KEY (student_id) REFERENCES student(student_id) ON DELETE CASCADE,
    FOREIGN KEY (leave_type_id) REFERENCES leave_type(leave_type_id)
);
```

```
CREATE TABLE leave_approval (
    approval_id INT AUTO_INCREMENT PRIMARY KEY,
    leave_id     INT NOT NULL,
    approver_id  INT NOT NULL,
    level        ENUM('Mentor','HOD') NOT NULL,
    decision     ENUM('Approved','Rejected') NOT NULL,
    remarks      VARCHAR(255),
    decided_on   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (leave_id, level),
    FOREIGN KEY (leave_id) REFERENCES leave_request(leave_id) ON DELETE CASCADE,
    FOREIGN KEY (approver_id) REFERENCES faculty(faculty_id)
);
```

7.2 Views

The view `v_attendance_percentage` calculates attendance for each student and course. Sessions with status On Leave are removed from the count, so approved leave never lowers the percentage.

```
CREATE VIEW v_attendance_percentage AS
SELECT s.student_id, s.roll_no, s.full_name, c.course_code,
       SUM(a.status = 'Present') AS attended,
```

```

SUM(a.status <> 'On Leave') AS conducted,

ROUND(100 * SUM(a.status = 'Present')

/ NULLIF(SUM(a.status <> 'On Leave'),0), 2) AS attendance_pct

FROM attendance a

JOIN student s          ON s.student_id = a.student_id

JOIN class_session cs ON cs.session_id = a.session_id

JOIN course c           ON c.course_id = cs.course_id

GROUP BY s.student_id, s.roll_no, s.full_name, c.course_id, c.course_code;

```

$\text{Attendance \%} = \text{Present sessions} \div (\text{Total sessions} - \text{On Leave sessions}) \times 100.$

A second view, v_pending_leaves, lists requests that are waiting for a mentor or HOD decision.

7.3 Stored procedure: applying for leave

The procedure sp_apply_leave refuses a request whose dates are reversed, whose dates overlap an existing active request of the same student, or which would take the student over the semester limit for that leave type. Only a valid request is inserted.

```

CREATE PROCEDURE sp_apply_leave(

    IN p_student_id INT, IN p_leave_type_id INT,

    IN p_from DATE, IN p_to DATE, IN p_reason VARCHAR(255))

BEGIN

    DECLARE v_used INT DEFAULT 0;  DECLARE v_max INT;  DECLARE v_req INT;

    IF p_to < p_from THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'To-date cannot be before from-date';

    END IF;

    IF EXISTS (SELECT 1 FROM leave_request

                WHERE student_id = p_student_id

                   AND status IN ('Pending', 'Mentor Approved', 'Approved')

                   AND p_from <= to_date AND p_to >= from_date) THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Overlapping leave request exists';

    END IF;

    SET v_req = DATEDIFF(p_to, p_from) + 1;

    SELECT max_days_per_sem INTO v_max FROM leave_type WHERE leave_type_id =

p_leave_type_id;

    SELECT COALESCE(SUM(DATEDIFF(to_date, from_date) + 1), 0) INTO v_used

        FROM leave_request

```

```

WHERE student_id = p_student_id AND leave_type_id = p_leave_type_id

AND status IN ('Approved', 'Mentor Approved', 'Pending');

IF v_used + v_req > v_max THEN

    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Leave balance exceeded for this
leave type';

END IF;

INSERT INTO leave_request(student_id, leave_type_id, from_date, to_date, reason)

VALUES (p_student_id, p_leave_type_id, p_from, p_to, p_reason);

END

```

7.4 Triggers: enforcing the approval order

The BEFORE INSERT trigger on leave_approval blocks decisions that are out of order, for example an HOD decision before the mentor has approved. The AFTER INSERT trigger then updates the request status and, on final approval, changes attendance.

```

CREATE TRIGGER trg_approval_before_insert BEFORE INSERT ON leave_approval

FOR EACH ROW

BEGIN

    DECLARE v_status VARCHAR(20);

    SELECT status INTO v_status FROM leave_request WHERE leave_id = NEW.leave_id;

    IF v_status IN ('Approved', 'Rejected', 'Cancelled') THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Leave request is already closed';

    END IF;

    IF NEW.level = 'Mentor' AND v_status <> 'Pending' THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Mentor decision already recorded';

    END IF;

    IF NEW.level = 'HOD' AND v_status <> 'Mentor Approved' THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'HOD can act only after mentor
approval';

    END IF;

END

```

```

CREATE TRIGGER trg_approval_after_insert AFTER INSERT ON leave_approval

FOR EACH ROW

BEGIN

    IF NEW.decision = 'Rejected' THEN

        UPDATE leave_request SET status = 'Rejected' WHERE leave_id = NEW.leave_id;

    END IF;

END

```

```
ELSEIF NEW.level = 'Mentor' THEN

    UPDATE leave_request SET status = 'Mentor Approved' WHERE leave_id =
NEW.leave_id;

ELSE

    UPDATE leave_request SET status = 'Approved' WHERE leave_id = NEW.leave_id;

    UPDATE attendance a

    JOIN class_session cs ON cs.session_id = a.session_id

    JOIN leave_request lr ON lr.leave_id = NEW.leave_id

                        AND lr.student_id = a.student_id

    SET a.status = 'On Leave'

    WHERE cs.session_date BETWEEN lr.from_date AND lr.to_date;

END IF;

END
```

7.5 Indexes

Indexes were created on attendance(student_id), leave_request(student_id, status), and class_session(session_date), because attendance reports and leave lookups filter on these columns.

8. Workflow of the System

1. The student applies for leave. The request is validated and saved with status Pending.
2. The mentor reviews the request. If rejected, the request is closed with status Rejected. If approved, the status becomes Mentor Approved.
3. The HOD reviews the request. The database allows this step only after mentor approval. If rejected, the request is closed.
4. On HOD approval the status becomes Approved, and every attendance record of that student within the leave dates changes to On Leave.
5. The attendance percentage is recalculated automatically by the view, excluding On Leave sessions.

The student can cancel a request while it is Pending or Mentor Approved. Every decision is saved in leave_approval with the approver, level, remarks and time, which gives a complete audit trail.

9. User Interface

A web interface demonstrates the workflow. A user chooses who to log in as: a student or a faculty member, and sees only the screens for that role. The interface applies the same rules as the database: overlap and balance checks on leave applications, mentor-before-HOD ordering, and automatic On Leave marking.

Role	Screens
Student	My Attendance (percentage per course, shortage warning, session log); Apply Leave; My Leaves (status, approval trail, cancel)
Faculty	Mark Attendance; Leave Approvals (approve or reject with remarks); Attendance Report with shortage highlighted

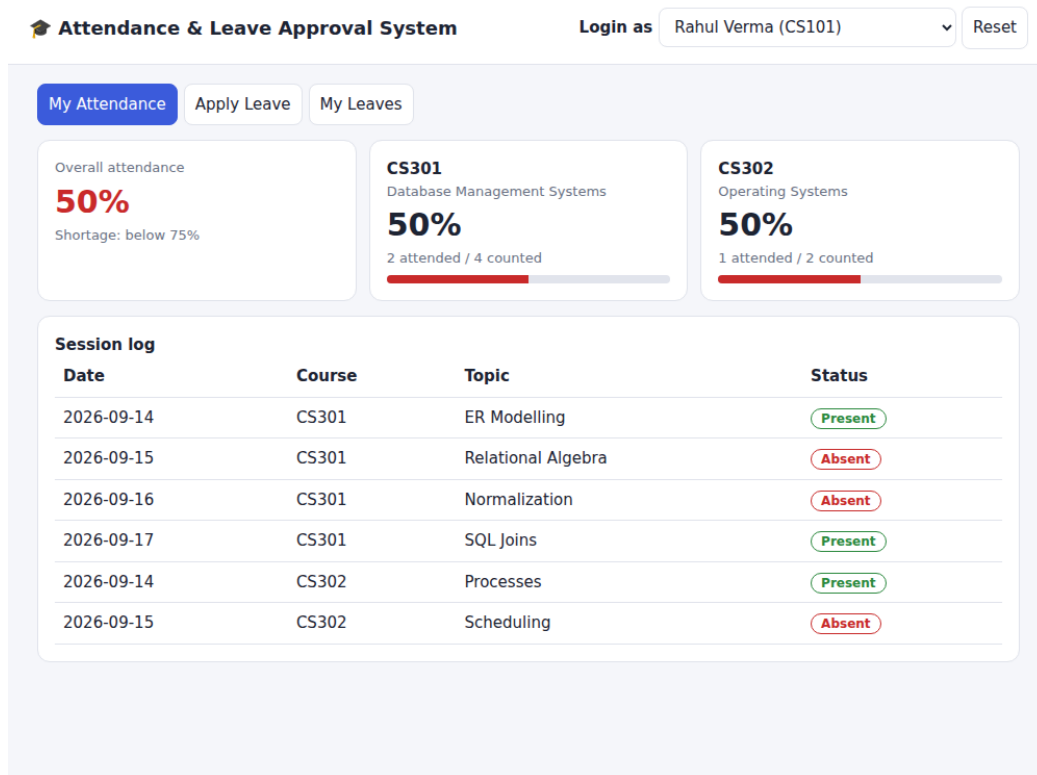


Figure 9.1: Student view of attendance before leave approval

Attendance & Leave Approval System

Login as

Rahul Verma (CS101)

Reset

My Attendance

Apply Leave

My Leaves

New leave request

Leave type

Medical (max 10 days)

From

mm/dd/yyyy

To

mm/dd/yyyy

Reason

Submit request

Figure 9.2: Leave application form

Attendance & Leave Approval System

Login as

Prof. Kiran Das (Mentor)

Reset

Mark Attendance

Leave Approvals

Attendance Report

Awaiting your decision

#	Student	Type	Dates	Days	Reason	Your level	Remarks
1	Rahul Verma	Medical	2026-09-15 → 2026-09-16	2	Fever, doctor advised rest	Mentor	optional

All leave requests

#	Student	Dates	Status	Trail
1	Rahul Verma	2026-09-15 → 2026-09-16	Pending	No decisions yet

Figure 9.3: Mentor’s approval queue

Attendance & Leave Approval System Login as Dr. Anita Rao (HOD) Reset

[Mark Attendance](#)
[Leave Approvals](#)
[Attendance Report](#)

Awaiting your decision

#	Student	Type	Dates	Days	Reason	Your level	Remarks
1	Rahul Verma	Medical	2026-09-15 → 2026-09-16	2	Fever, doctor advised rest	HOD	optional Approve

All leave requests

#	Student	Dates	Status	Trail
1	Rahul Verma	2026-09-15 → 2026-09-16	Mentor Approved	Mentor: Approved by Prof. Kiran Das (Mentor)

Leave approved

Figure 9.4: HOD's queue after the mentor has approved

Attendance & Leave Approval System Login as Rahul Verma (CS101) Reset

[My Attendance](#)
[Apply Leave](#)
[My Leaves](#)

Overall attendance

100%

Meets 75% requirement

CS301
Database Management Systems

100%

2 attended / 2 counted

CS302
Operating Systems

100%

1 attended / 1 counted

Session log

Date	Course	Topic	Status
2026-09-14	CS301	ER Modelling	Present
2026-09-15	CS301	Relational Algebra	On Leave
2026-09-16	CS301	Normalization	On Leave
2026-09-17	CS301	SQL Joins	Present
2026-09-14	CS302	Processes	Present
2026-09-15	CS302	Scheduling	On Leave

Leave approved - attendance updated to On Leave

Figure 9.5: Student view after final approval; the leave days now show On Leave

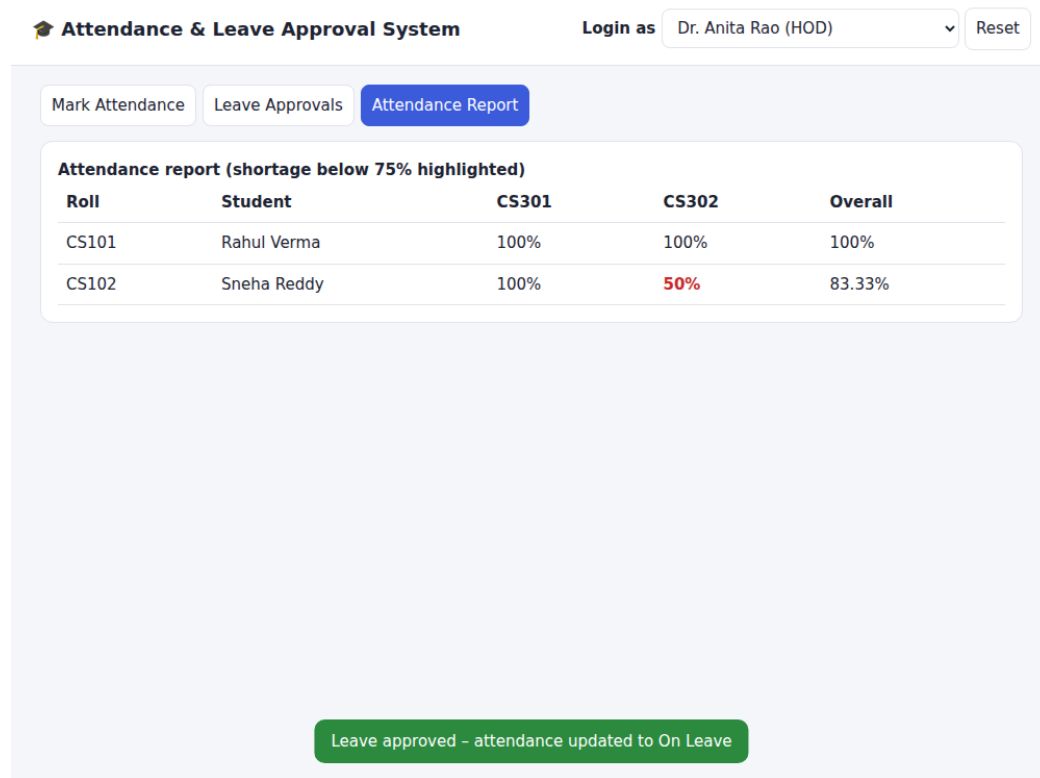


Figure 9.6: Faculty attendance report

The interface is a front-end demonstration. It keeps its sample data in the browser and is not connected to the MySQL server; the MySQL script is the database implementation.

10. Testing and Results

The sample data has three students, three faculty members, two courses, six class sessions, and three leave types. Rahul Verma (CS101) is absent on 15 and 16 September and applies for medical leave covering those dates. The tests below check the rules of the system.

No	Test	Expected result	Rule checked
1	Apply leave with to-date before from-date	Rejected: “To-date cannot be before from-date”	CHECK / procedure
2	Apply leave overlapping an existing active request	Rejected: “Overlapping leave request exists”	Overlap check
3	Apply 11 days of Medical leave (limit 10)	Rejected: “Leave balance exceeded”	Balance check
4	HOD decides before the mentor	Rejected: “HOD can act only after mentor approval”	BEFORE trigger
5	Mentor approves	Status changes Pending → Mentor Approved	AFTER trigger
6	Mentor rejects	Status changes to Rejected; request closed	AFTER trigger
7	HOD approves after mentor	Status Approved; attendance on leave dates becomes On Leave	AFTER trigger
8	Mark same student twice for a session	Rejected by UNIQUE (session_id, student_id)	Unique constraint
9	Decide on an already closed request	Rejected: “Leave request is already closed”	BEFORE trigger

10.1 Result with sample data

Rahul attended 2 of 4 DBMS sessions, so his DBMS attendance was 50%. After the mentor and the HOD approved his leave, the two absent sessions changed to On Leave and were removed from the count, so his attendance became 2 of 2, or 100%. Sneha Reddy’s leave was rejected by the mentor, so her record is unchanged.

Student (DBMS course)	Before approval	After approval	Reason
Rahul Verma	50%	100%	Two absences became On Leave
Sneha Reddy	100%	100%	Leave rejected; no change

The report queries (Appendix A) were used to list students with shortage, the approval trail of each request, and leave usage against the semester limit. The screenshots in Section 9 show the same behaviour in the interface.

11. Conclusion and Future Scope

This project built a normalized relational database that links attendance and leave in one system. Because the rules are written as constraints, a stored procedure, and triggers, they apply however the data is entered. Approved leave no longer causes false attendance shortages, every decision has an audit trail, and invalid or overlapping leave is stopped before it is stored.

Possible extensions:

- Connect the interface to MySQL through a back-end such as Flask or Java Servlets, with secure login and role-based access.
- Send email or SMS notifications when a request is approved or attendance falls below 75%.
- Add QR-code or biometric attendance marking.
- Add a document upload for medical certificates and an academic-calendar table for holidays.
- Add semester-wise archiving and graphical dashboards for administrators.

References

1. Silberschatz, A., Korth, H. F., and Sudarshan, S. Database System Concepts. McGraw-Hill.
2. Elmasri, R. and Navathe, S. B. Fundamentals of Database Systems. Pearson.
3. Oracle Corporation. MySQL 8.0 Reference Manual: CREATE TRIGGER, CREATE PROCEDURE, SIGNAL, and CHECK constraints. dev.mysql.com/doc.
4. Course lecture notes, Database Management Systems, Woxsen University.

Appendix A: Report Queries

Query 1: attendance percentage per student per course.

```
SELECT * FROM v_attendance_percentage ORDER BY roll_no, course_code;
```

Query 2: students with a shortage (below 75%) in any course.

```
SELECT roll_no, full_name, course_code, attendance_pct  
  
FROM v_attendance_percentage  
  
WHERE attendance_pct < 75;
```

Query 3: leave requests with their approval trail.

```
SELECT lr.leave_id, s.full_name, lt.type_name, lr.from_date, lr.to_date,  
  
       lr.status, la.level, f.full_name AS approver, la.decision, la.remarks  
  
FROM leave_request lr  
  
JOIN student s      ON s.student_id = lr.student_id  
  
JOIN leave_type lt  ON lt.leave_type_id = lr.leave_type_id  
  
LEFT JOIN leave_approval la ON la.leave_id = lr.leave_id  
  
LEFT JOIN faculty f      ON f.faculty_id = la.approver_id  
  
ORDER BY lr.leave_id, la.decided_on;
```

Query 4: approved leave days used against the semester limit.

```
SELECT s.full_name, lt.type_name,  
  
       SUM(DATEDIFF(lr.to_date, lr.from_date) + 1) AS days_used, lt.max_days_per_sem  
  
FROM leave_request lr  
  
JOIN student s      ON s.student_id = lr.student_id  
  
JOIN leave_type lt  ON lt.leave_type_id = lr.leave_type_id  
  
WHERE lr.status = 'Approved'
```

```
GROUP BY s.student_id, s.full_name, lt.leave_type_id, lt.type_name, lt.max_days_per_sem;
```

Query 5: pending leave requests for a given mentor.

```
SELECT * FROM v_pending_leaves WHERE mentor_id = 2;
```

Query 6: number of leave requests rejected by each mentor.

```
SELECT f.full_name AS mentor, COUNT(*) AS rejected  
  
FROM leave_approval la JOIN faculty f ON f.faculty_id = la.approver_id  
  
WHERE la.level = 'Mentor' AND la.decision = 'Rejected'  
  
GROUP BY f.faculty_id, f.full_name;
```